

EGN4950C Senior Design

Bi-Weekly Progress Report, Report 3

Project Title: Open Source Selective Video Compression for Static Surveillance Cameras

Technical Sponsor: Defense Innovation Unit (DIU) / NIWC Pacific, Cody Hayashi

Report Period: April 15, 2026 to April 26, 2026

Submission Date: April 26, 2026

Team Members: Kheiven D'Haiti (CS, AI Minor) | Jorge Sanchez (CS) | Ashleyn Montano (CS) | Riley Roberts (CS) | Victor De Souza Teixeira (CS, Cybersecurity)

Part 1: Team Section

1. Team Meeting

Did the team meet this period? No

No team meeting this period. Coordination happened through GitHub PRs, branch reviews, and direct messages. The April 15 meeting listed as the next meeting in Report 2 did not happen — we coordinated the Milestone 3 sprint asynchronously. Closing PRs #12 and #13 and landing the YOLO gate were tracked through GitHub comments and direct messages.

2. Sponsor / Advisor Meeting

Did the team meet with the sponsor this period? No

No sponsor meeting this period. The April 8 meeting covered in Report 2 was the last formal check-in. The next sponsor-facing milestone is the final demo.

3. Team Progress This Period

Four open branches were merged and two pull requests were closed this period. Victor's AES-256-GCM encryption upgrade (PR #12) is in and replaces the old CBC implementation — the new format adds a 128-bit authentication tag that catches any tampering before plaintext is returned. Riley's Mode 2 and Mode 3 implementation (PR #11) landed after the streaming encoder refactor broke the mode dispatch; both modes are now live and selectable in the GUI. Ashleyn's multi-type metadata query branch (m3-metadata-query-fix) extended the database query interface to accept compound object-type filters and a minimum ROI count threshold. Jorge's watchfolder daemon and MultiFrameSource (PR #13) are in, covering auto-ingestion of drop-folder footage and parallel RTSP stream management.

Kheiven landed the YOLO object classification gate (PR #31), which runs YOLOv8-nano on each bounding box crop produced by background subtraction before the frame reaches the encoder. Testing on nightVideos_busyBoulevard.mp4 identified conf=0.10 as the practical

threshold for night scenes (184 frames passed vs. 2 at the default conf=0.30). The full test suite was repaired to match the streaming encoder API: 274 tests pass, 0 fail.

4. Challenges and Blockers

- The streaming encoder API — `begin_segment()` → `write_frame()` × N → `finish_segment()` — had been landed in a prior session but none of the test mocks had been updated. All four dummy encoder classes in `test_pipeline.py` still used the old `encode_segment(frames, bboxes_per_frame, ...)` signature and were compiling and running without actually hitting the real code path. All four were rewritten.
- `finish_segment()` returns a dict with keys `file_path`, `avg_sharpness`, and `sharpness_label`. Multiple test files were still treating the return value as a bare path string. Fixed in `test_roi_encoder.py` and `test_data_integrity.py`.
- `test_object_type_queries.py` was accessing `results[0][-1]` expecting `object_type`, but the schema had grown three new columns (`avg_sharpness`, `sharpness_label`, `hidden`) since that index was written, pushing it off the end. Fixed with an explicit `_OBJECT_TYPE_COL = 8` constant.
- The `test_any_alive_true_when_running` test in `test_multi_source.py` was racy. The mock stream used `frame_count=500`, but the daemon thread could exhaust all 500 frames before the main thread reached the assertion. Fixed by blocking `read()` with a `threading.Event` gate until after the assertion.
- Night scene YOLO confidence: conf=0.30 (the default) passes only 2 frames on `nightVideos_busyBoulevard.mp4` because distant vehicles at night are classified below that threshold. conf=0.10 passes 184. Night mode users need `--yolo-conf 0.10` for now; a GUI toggle is planned before the final demo.
- AES-256-GCM is not backward-compatible with the old CBC format. Existing `.enc` files cannot be decrypted with the new code. The team was notified — re-encrypt any stored segments before the final demo.

5. System Architecture Overview

5a. Block diagram: components and data flow

Seven stages. Camera or drop-folder footage in, compressed, encrypted, and indexed video out.

Stage	What it does
1. Frame ingest	Reads frames from a video file, live RTSP stream, or watchfolder drop via OpenCV VideoCapture. Converts to BGR numpy arrays.
2. Background subtraction	Runs each frame through MOG2 (or KNN). Returns a foreground mask and bounding boxes around moving objects. CLAHE preprocessing activates in night mode.

3. YOLO filter gate	Optional. Runs YOLOv8-nano on each bounding box crop. Rejects crops whose top class is not in the object allowlist. A static suppression grid suppresses persistent false-positive cells.
4. Mode filter	Mode 0: all frames pass. Mode 1: only frames with detections are buffered. Mode 2: object patches composited over a background keyframe. Mode 3: everything outside bounding boxes is blacked out.
5. Optional enhancement	If --enhance is set, each detected ROI is passed through the Enhancer class. Real-ESRGAN upscales the region; bicubic fallback is used when weights are absent. CUDA and MPS acceleration now available.
6. ROI encoding	Frames are piped to FFmpeg one at a time via the streaming API (begin_segment / write_frame / finish_segment). Segments with detections encode at CRF 18. Background-only segments encode at CRF 45.
7. Metadata + encryption	One row written to SQLite per encoded segment. If --encrypt is set, the output file is encrypted with AES-256-GCM before being written to disk.

5b. Subsystem table

Subsystem	Purpose	Owner
Background subtraction	MOG2/KNN foreground detection with night mode, morphological cleanup, and min-area filter	Kheiven D'Haiti
YOLO filter gate	YOLOv8-nano classification of MOG2 bounding box crops; static suppression grid for persistent false-positive cells	Kheiven D'Haiti
Pipeline orchestrator	Coordinates frame ingest, mode filtering, YOLO gate, enhancement, streaming encode, and database writes	Kheiven D'Haiti
ROI encoder (streaming)	Streaming H.264 MP4 encoder via FFmpeg subprocess: begin_segment / write_frame / finish_segment API	Jorge Sanchez
Stress test + algorithm comparison	1-hour simulated stress test, MOG2 vs KNN benchmark across 46 CDnet scenes and 30 parameter combinations	Jorge Sanchez
Watchfolder + MultiFrameSource	Drop-folder auto-ingestion daemon and parallel RTSP stream manager with stall detection	Jorge Sanchez
Compression modes 2 and 3	Background keyframe + object patch encoding (Mode 2) and object-only forensic encoding (Mode 3)	Riley Roberts

Metadata database	SQLite index of every encoded segment; multi-type queries with ROI count filter; CLI tool	Ashleyn Montano
AES-256-GCM encryption	Authenticated encryption for stored segments; GCM tag detects tampering before any plaintext is returned	Victor De Souza Teixeira
Enhancement module	CPU/CUDA/MPS super-resolution via Real-ESRGAN with bicubic fallback; detect_gpu() for capability reporting	Victor De Souza Teixeira
Web dashboard	Browser-based operator interface with live log streaming, pipeline controls, HLS live view, and segment query archive	Kheiven D'Haiti

6. Summary Tables

Previous report period: March 30 to April 15, 2026 (Report 2)

Current report period: April 15 to April 26, 2026:

Member	Tasks completed	Not completed	Tasks for next period
Kheiven D'Haiti	5	0	4
Jorge Sanchez	2	0	2
Ashleyn Montano	1	0	2
Riley Roberts	2	0	2
Victor De Souza Teixeira	2	0	2

Part 2: Individual Sections

Each team member writes their own section independently. Task numbers match MS Teams Planner.

Kheiven D'Haiti, CS Major, AI Minor

Task 3.1a: YOLO object classification gate — PR #31 (due: April 24)

Description: I built the YOLO filter gate as a new ObjectFilter class in src/detection/object_filter.py. The class wraps YOLOv8-nano (ultralytics) and runs on each bounding box crop produced by MOG2 before the frame reaches the encoder. Each crop is classified at a configurable confidence threshold (default conf=0.30). Crops whose top predicted class is not in the allowlist — person, car, truck, bus, bicycle, motorcycle, dog, bird — are

rejected before the frame is buffered. I also built a 32-pixel static suppression grid using int16 counters: cells that receive 30 consecutive false-positive-only frames are suppressed for the rest of the session, which handles persistent leaf and shadow hotspots without requiring per-scene MOG2 retuning. The gate is fully optional — if ultralytics is not installed, the pipeline logs a warning and runs without filtering.

I ran a five-run calibration series on nightVideos_busyBoulevard.mp4 to find the right confidence threshold for night footage:

conf	Frames passed	Notes
0.70	0	Rejects everything at night
0.30	2	Misses most distant vehicles
0.10	184	Practical sweet spot for low-light

Outcome: PR #31 merged. The default of conf=0.30 is appropriate for daytime scenes. Night and low-light scenes need --yolo-conf 0.10. A GUI toggle is planned before the final demo. 274 tests pass after the gate integration.

Task 3.1b: Repair test suite for streaming encoder API (due: April 24)

Description: The streaming encoder API — begin_segment() -> write_frame() x N -> finish_segment() — had been landed in a prior session but none of the test mocks in test_pipeline.py had been updated. All four dummy encoder classes still used the old encode_segment(frames, bboxes_per_frame, ...) signature. The tests compiled and ran but were not exercising the real code path. I rewrote all four classes to implement the full streaming interface, with finish_segment() returning the expected dict format (file_path, avg_sharpness, sharpness_label). The Mode 2 test required special attention: it captures background_frame and object_only per write_frame() call rather than from a single kwargs dict. I also fixed test_roi_encoder.py and test_data_integrity.py (return value changed from string to dict) and test_object_type_queries.py (replaced results[0][-1] with _OBJECT_TYPE_COL = 8 after schema additions broke the old index).

Outcome: 274 passed, 0 failed.

Task 3.1c: GPU support for enhancer.py (due: April 22)

Description: I extended src/enhancement/enhancer.py to support CUDA and Apple Silicon MPS in addition to CPU. Device selection is automatic: CUDA first, then MPS, then CPU. I also wrote a detect_gpu() utility that returns a dict with backend, device_name, cuda_available, mps_available, vram_mb, and will_work fields. This is exposed via /api/gpu_info so the dashboard can report what acceleration is available on startup. The pipeline falls back to bicubic interpolation when model weights are missing or the device fails to initialize — no behavior change for deployment on government COTS hardware.

Outcome: On an M2 MacBook Pro, MPS reduced per-frame enhancement time from approximately 420 ms to 95 ms. CPU-only hardware is unaffected.

Task 3.1d: Session log and CRLF normalization (due: April 26)

Description: Wrote docs/session_log_2026-04-26.md covering all M3 work from April 20 through April 26. Normalized CRLF line endings to LF across all docs, notebooks, scripts, and source files (40 files) for consistent cross-platform git diffs.

Outcome: Session log committed. Working tree clean.

Task 3.1e: Mode label overlay and double-compositing fix (due: April 20)

Description: Added a mode label and elapsed timer overlay burned into every output segment via cv2.putText() inside write_frame() before piping to FFmpeg. Also removed a double-compositing bug where the background composite was being applied both inside write_frame() and in the outer loop in pipeline.py, causing Mode 2 segments to show a washed-out background artifact.

Outcome: Both commits in dev (5547564, 39fef25). Mode label is visible in archived .mp4 files and the HLS stream.

Planned tasks for next period:

- Task 3.5a: Test pipeline on USB or IP camera input in real time — confirm no frame drops on target hardware (due: April 30)
- Task 3.5b: Research deployment packaging for government COTS hardware — Docker, PyInstaller, source tarball options (due: April 30)
- Task 3.6a: Create final_results.ipynb — reproducible end-to-end benchmark notebook (due: May 2)
- Task 3.8a: Slide deck — problem, approach, four-mode system, benchmark results, demo footage (due: April 28)

Jorge Sanchez, CS Major

Task 3.3a: Watchfolder daemon — src/utils/watchfolder.py (due: April 24)

Description: I built the watchfolder daemon in src/utils/watchfolder.py. It polls a configurable drop folder every 5 seconds and detects new video files by extension (.mp4, .avi, .mov, .mkv, .ts, .mts, .m2ts). For each new file it calls _is_fully_written() — a size-stability check that waits for the file to stop growing before ingesting — then calls run_pipeline() on it. A .ingested sentinel file is written beside each processed video to prevent double-processing after a restart. The daemon also builds a sanitized camera ID from the filename and folder path using _sanitize_camera_id(), which applies the same character-allowlist logic as the main pipeline.

Outcome: PR #13 merged. 22 tests across test_watchfolder.py covering ID sanitization, the sentinel mechanism, file stability check, and scan-and-ingest behavior including dry-run and already-ingested skipping.

Task 3.3b: MultiFrameSource — src/utils/multi_source.py (due: April 24)

Description: I built MultiFrameSource in src/utils/multi_source.py. The class manages N parallel RTSP streams via _StreamReader daemon threads. Each reader maintains a 2-frame ring buffer and monitors the stream for stalls using a FRAME_TIMEOUT of 5.0 seconds. Public API: open(), read_all(), any_alive(), active_count(), get_metadata(), release(), and a context manager. The test any_alive_true_when_running test had a race condition — the mock used frame_count=500 but the background thread could exhaust all 500 before the main thread reached the assertion. Fixed by replacing the mock with a threading.Event gate that holds read() open until after the assertion completes.

Outcome: PR #13 merged. Both modules in dev. Race condition fixed and documented.

Planned tasks for next period:

- Task 3.5b: Run 1080p CDnet footage benchmark to establish real storage upper bound for high-resolution cameras (due: May 2)
- Task 3.8a: Contribute demo segment and slide content for the capstone presentation (due: April 28)

Ashleyn Montano, CS Major

Task 3.2a: Multi-type metadata query — branch m3-metadata-query-fix (due: April 22)

Description: I extended query_by_type() in src/utils/db.py to accept either a single string or a list of strings as object_type. When a list is passed, the query uses parameterized IN (?, ?, ...) placeholders — not string interpolation — so SQL injection safety is maintained. I also added a min_roi_count parameter that filters out low-confidence detections. The CLI tool in src/utils/db_query.py was rewritten at the same time: --type now accepts multiple values in one call, --min-roi filters by minimum ROI count, and --last-hours generates start and end timestamps automatically using timezone-aware UTC datetimes. The old CLI only supported single-type queries with no count filtering.

Outcome: Branch merged to dev via commit ba4f3db. Operators can now run queries like: python -m src.utils.db_query --type vehicle person --last-hours 12 --min-roi 5

Planned tasks for next period:

- Task 3.4a: Full-text and multi-tag search in the query archive sidebar — extend the GUI query interface to match the new multi-type CLI (due: May 2)
- Task 3.4b: Write integration test that runs the pipeline on a short clip and verifies object_type is populated for all segments in the resulting database (due: May 2)

Riley Roberts, CS Major

Task 2.3a-b: Mode 2 and Mode 3 implementation — PR #11 (due: April 20)

Description: I completed Mode 2 and Mode 3 and merged them via the fix/repair-dev-mode-system branch. Mode 2 captures one clean background frame during warmup — or falls back to the last warmup frame if no fully static frame appears — then composites per-frame object patches over it. The result is a segment that encodes only the moving content on top of a frozen reference background. Mode 3 passes `object_only=True` to ROIEncoder, which blacks out all pixels outside detected bounding boxes before piping to FFmpeg. Both modes are integrated with the GUI mode selectors and work correctly with the streaming encoder API. The branch also repaired the mode dispatch in `modes.py` that had broken during the streaming encoder refactor — modes were silently falling back to Mode 0 behavior before this fix.

Outcome: PR #11 merged. Mode 0, 1, 2, and 3 all pass their respective test classes in `test_pipeline.py`. Mode label overlay confirms which mode is active in every output segment.

Task 3.2b: Mode system repair and streaming API integration (due: April 20)

Description: The streaming encoder refactor had left `modes.py` referencing the old `encode_segment()` call signature. I updated the pipeline to split `raw_regions` (pre-YOLO, used for background keyframe tracking in Mode 2) from `regions` (post-YOLO, used for encoder decisions). This ensures Mode 2 background selection uses the full unfiltered region list while the encoder only sees YOLO-validated detections.

Outcome: Committed as part of PR #11. Background keyframe fallback works correctly on dynamic scenes where no fully-static warmup frame exists.

Planned tasks for next period:

- Task 3.6b: Demo/concat mode — stitch all output segments into one reviewable file so operators can review a full session without opening each 60-second clip individually (due: May 2)
- Task 3.8a: Prepare 2-minute live demo segment for the capstone presentation (due: April 29)

Victor De Souza Teixeira, CS Major, Cybersecurity

Task 3.1a: AES-256-GCM encryption upgrade — PR #12 (due: April 20)

Description: I upgraded `src/utls/encryption.py` from AES-256-CBC to AES-256-GCM. CBC has no authentication tag, which means a bit-flip attack can silently corrupt a stored video segment with no detection. GCM adds a 128-bit authentication tag: any modification to the ciphertext, even a single bit, raises `InvalidTag` before any plaintext is returned. The new file format stores the nonce (12 bytes), salt (16 bytes, zeros in raw-key mode), auth tag (16 bytes), and ciphertext in sequence. The public API is unchanged — `encrypt_file()` and `decrypt_file()` work the same

way for callers. I also added `encrypt_bytes()` and `decrypt_bytes()` for in-memory use, which the pipeline uses when packaging segments without writing plaintext to disk first. The test suite covers 1 KB, 1 MB, and multi-chunk round-trips, plus a tamper detection test that modifies one byte of ciphertext and verifies `InvalidTag` is raised.

Outcome: PR #12 merged. 24 unit tests in `test_encryption.py`, all passing. Note for the team: existing `.enc` files encrypted under the old CBC format cannot be decrypted with the new code. Re-encrypt any stored segments before the final demo.

Task 3.1b: GPU device detection for enhancer.py (due: April 22)

Description: I added the `detect_gpu()` and `best_device()` utility functions to `src/enhancement/enhancer.py`. `detect_gpu()` probes the system for CUDA and MPS support and returns a structured dict with `backend`, `device_name`, `cuda_available`, `mps_available`, `vram_mb`, `torch_version`, `will_work`, and a human-readable note field. `best_device()` returns the highest-capability available backend as a string. Both are used by the new `/api/gpu_info` endpoint so the dashboard can surface acceleration status on startup.

Outcome: Committed as part of the CRLF normalization pass (30b10d4). Dashboard GPU info endpoint live.

Planned tasks for next period:

- Task 3.2c: Store IV and salt in the database per segment so segments can be decrypted individually without re-prompting the user for their password on each one (due: May 2)
- Task 3.2d: Build password-protected incident clip export — encrypt a flagged clip with a user-supplied passphrase for secure handoff to investigators (due: May 2)